İleri Python Programlama – Hafta 4 Lab Ödevi

Üreteçler ve functools

Ders: İleri Python Programlama · **Kurum:** İstanbul Nişantaşı Üniversitesi · **Öğretim üyesi:** Dr. Öğr. Üyesi Murat Emeç · **Dönem:** 2026–2027 Güz

Öğrenci: FATİH YUMUŞAK **Numara:** 20253006032 **GitHub deposu:** Fatih-Yumusak/python_ile_programlama_odev

Bu lab, uzay projesine üreteçler, bir veri işleme boru hattı ve önbellekli bir rota hesabı ekler. Hafta 3'ün son commit'inden devam edin. Yeni kodu `uzay_oyunu/akis.py` modülünde toplayın; her fonksiyon tip ipucu ve docstring içersin, her görev için `tests/test_akis.py` dosyasına test yazın.

Görev 1 – Fibonacci üreteci (15 puan)

Sonsuz bir `fibonacci()` üreteci yazın. Değerleri üç farklı yolla alın: `next()` ile ilk üç değeri, `itertools.islice` ile ilk 12 değeri, `itertools.takewhile` ile 100'den küçük değerleri. Testte üretecin **tek kullanımlık** olduğunu da gösterin: aynı üreteçten ikinci `list(islice(...))` çağrısı farklı sonuç vermelidir.

```
>>> g = fibonacci(); next(g), next(g), next(g)
(0, 1, 1)
>>> list(islice(fibonacci(), 12))
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Görev 2 – Yakıt göstergesi (20 puan)

`yakit_gostergesi(yakit: int, tuketim: int)` üretecini yazın: her `next()` çağrısında yakıtı `tuketim` kadar azaltıp kalan yakıtı üretsin, yakıt 0'a inince dursun. `travel()` içinde oyunun başında galaksideki `yakit` değeriyle bir gösterge oluşturun ve **her yolculukta** bir değer çekin. `StopIteration` yakalanınca oyuncu "Yakıtın bitti, uzayda sürükleniyorsun…" mesajıyla oyunu kaybetsin. Mevcut testlerin bozulmaması için tüketimi yeterince küçük seçin (ör. yakıt 100, tüketim 5).

```
>>> list(yakit_gostergesi(100, 30))
[70, 40, 10, 0]
```

Görev 3 – Seyir defteri boru hattı (25 puan)

Aşağıdaki içerikle `veri/seyir.txt` dosyası oluşturun. Ardından üç üreteçli bir boru hattı yazın: `satirlari_oku(yol)` boş ve `#` ile başlayan satırları atlayarak satırları üretsin; `ayristir(satirlar)` her satırdan `(kaynak, hedef, yakit)` demeti üretsin; `uzun_yolculuklar(kayitlar, esik)` yakıtı eşikten büyük olanları süzsün. Hiçbir aşama tüm dosyayı bir listeye okumamalıdır.

```
dünya -> sirius 12
sirius -> orion 8
orion -> centauri 15
# bakım molası
centauri -> orion 15
orion -> sirius 8
sirius -> kara_delik 30
```

```
>>> sum(y for _, _, y in ayristir(satirlari_oku("veri/seyir.txt")))
88
>>> kayitlar = ayristir(satirlari_oku("veri/seyir.txt"))
```

```
>>> [h for _, h, _ in uzun_yolculuklar(kayitlar, 10)]
['sirius', 'centauri', 'orion', 'kara_delik']
```

Görev 4 – Önbellekli rota sayacı (25 puan)

`rota_sayaci(baglantilar)` fonksiyonu, içinde `@lru_cache` ile süslenmiş `rota_sayisi(gezegen, adim)` fonksiyonunu tanımlayıp döndürsün. `rota_sayisi`, verilen gezegenden başlayıp tam `adim` yolculuk yapan farklı rotaların sayısını hesaplasın. Önce önbelleksiz sürümün `adim=20` için kaç saniye sürdüğünü `time.perf_counter()` ile ölçün, sonra önbellekli sürümle karşılaştırın; iki süreyi ve `cache_info()` çıktısını README'ye yazın.

```
>>> say = rota_sayaci(galaksi["baglantilar"])
>>> say("dünya", 3), say("dünya", 10), say("dünya", 40)
(9, 2185, 16832258606399)
```

(Değerler Mars'sız galaksi için geçerlidir; Mars eklediyseniz sonuçlarınız farklı olur, testinizi kendi galaksinize göre yazın.)

Görev 5 – map, filter, reduce ve partial (15 puan)

`istatistik.py` modülünde, galaksi verisi üzerinde aynı üç soruyu **iki yolla** (önce `map`/`filter`/`reduce`, sonra liste üreteci veya `sum`) yanıtlayan fonksiyonlar yazın: gezegen adlarının büyük harfli listesi; en az iki bağlantısı olan gezegenler; toplam bağlantı sayısı. Ayrıca `functools.partial` ile `yolculuk` fonksiyonundan `hizli_yolculuk` adında, `hiz=3` sabit yeni bir fonksiyon türetin. Hangi yazımı daha okunaklı bulduğunuzu bir yorum satırında açıklayın.

```
['DÜNYA', 'CENTAURI', 'SIRIUS', 'ORION', 'KARA_DELIK']
['dünya', 'centauri', 'sirius', 'orion']
10
```

Bonus (+10). Kaynak kitaptaki `zamanlayici(aralik)` üretecini kullanarak, 10 saniye boyunca her 2 saniyede "tik", her 5 saniyede "tak" yazan bir program yazın. Neden bunun bir "meşgul bekleme" olduğunu ve `time.sleep` ile farkını açıklayın.

Teslim ve puanlama

- **Teslim:** GitHub deposunun bağlantısı, bir sonraki lab dersinden önce. Her görev en az bir commit.
- Tüm eski ve yeni testler `uv run pytest` ile geçmelidir.
- Kopya tespit edilen ödevler, veren ve alan için sıfır puan alır. Öğrenci her fonksiyonu açıklayabilmelidir.

Ölçüt	Pay	Tam puan için
Doğruluk	%50	Üreteçler ve fonksiyonlar örnek çıktıları veriyor, testler geçiyor
Tembellik ve verimlilik	%20	Boru hattı listeye okumuyor; önbellek doğru yerde
Testler	%15	Tek kullanımlık üreteç ve StopIteration testleri mevcut
Belgeleme	%15	Tip ipuçları, docstring'ler, README'deki ölçümler